1.4、自由存储区对象的构造与析构

类对象的动态分配与撤销

```
CGoods *pc;
pc=new CGoods;
.......
delete pc;
```

分配自由存储区空间，调用构造函数，构造一个无名的CGoods对象

调用析构函数析构，然后将内存空间返回给自由存储区

自由存储区类对象初始化

- 创建一个动态对象时，new后面类（class）类型可以有参数。这些参数即构造函数的参数。
- 创建对象数组，无参数，调用默认的构造函数。

注：vs2017以 上版本里是允许调用有参构造函数的

1.4、自由存储区对象的构造与析构

示例

```
class CGoods
{
    string  Name;
    int  Amount;
    float Price;  float Total value;
public:
    CGoods() {cout<<"调用默认构造函数"<<endl;}
    CGoods(string name,int amount ,float price)
    {
        cout<<"调用三参数构造函数"<<endl;
        Name=name;  Amount=amount;
        Price=price;  Total_value=price*amount;
     }
    ~CGoods(){ cout<<"调用析构函数"<<endl;}
};
```

1.4、自由存储区对象的构造与析构

示例

```
int main()
{
    int n;
    CGoods *pc,*pc1,*pc2;
    pc=new CGoods(“夏利2000”，10，118000);
    pc1=new CGoods;
    cout<<“输入商品类数组元素数” <<endl;
    cin>>n;
    pc2=new CGoods[n];
    delete pc;
    delete pc1;
    delete [ ] pc2;
    return 0;
}
```

调用三参数构造函数

Point (1, 4)　Point (3, 1)

调用默认构造函数

动态建立数组，调用n次默认构造函数

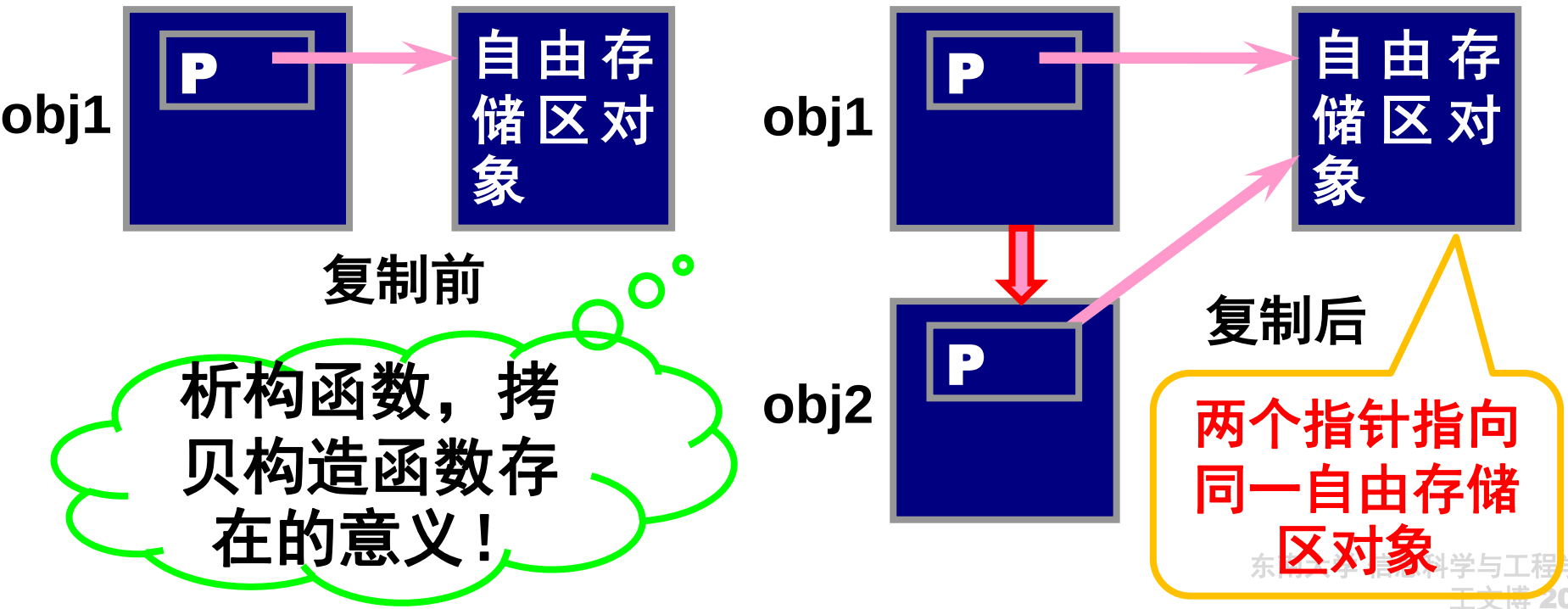
释放动态内存，调用析构函数，不要忘！！！

1.5、浅复制与深复制

浅复制：

默认复制构造函数，可用一个类对象初始化另一个类对象，称为默认的按成员复制。

存在的问题：

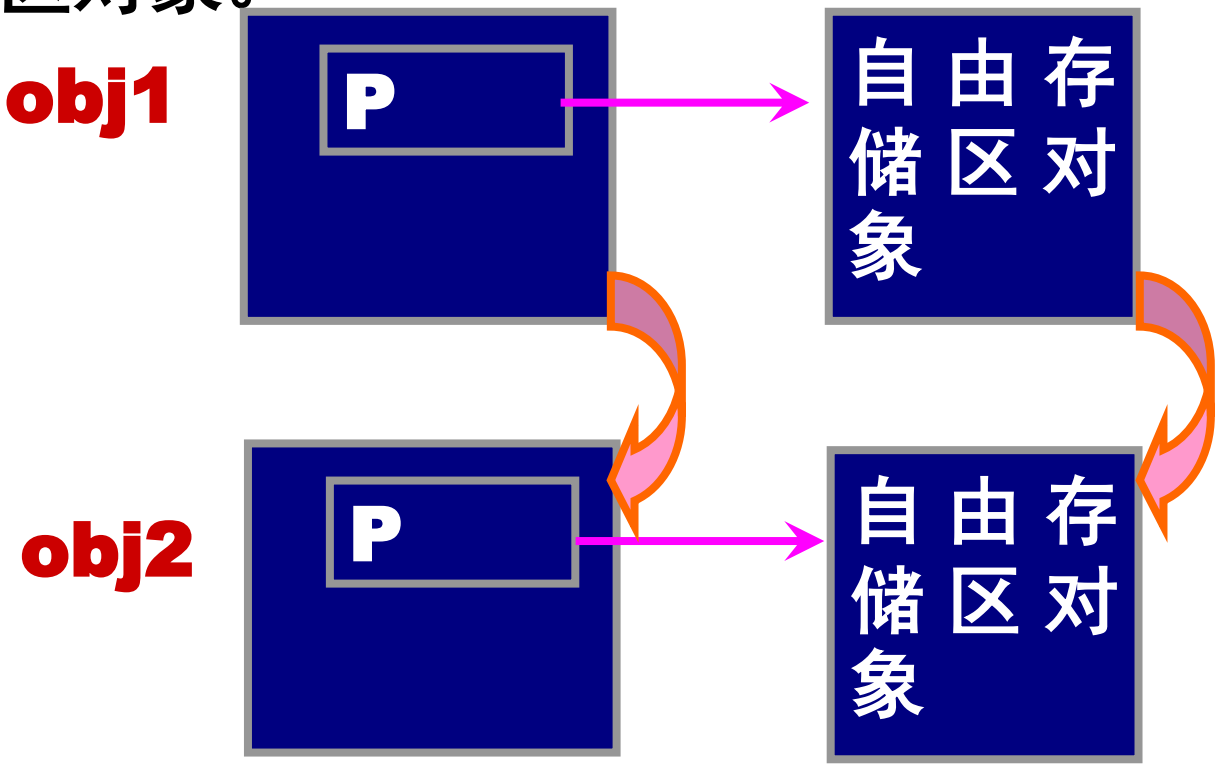


东南大学 吴健雄科学与工程学院
王文博 2026

1.5、浅复制与深复制

深复制：

重新定义复制的构造函数，给每个对象独立分配一个自由存储区对象。



1.5、浅复制与深复制

示例

```
class student
{
      char *pName;
public:
     student(); //默认构造函数
     student(char *pname); //带参数构造函数
     ~student(); //析构函数
     student(student &s); //复制构造函数
     student & operator=(student &s); //复制赋值操作符
};
```

检验主函数和运行结果

1.5、浅复制与深复制

无参构造函数

```
student :: student()
{
    cout<<"Constructor1";
    pName=NULL;
    cout<<“默认” <<endl;
}
```

有参构造函数

```
student::student(char *pname)
{
  cout<<"Constructor2";
  if(pName=new char[strlen(pname)+1])
      strcpy(pName,pname);
  cout<<pName<<endl;
}
```



1.5、浅复制与深复制

析构函数

```
student :: ~student()
{
    cout<<“Destructor” ;
    if(pName)
    {
        cout<<pName<<endl;
        delete[] pName; //释放字符串
    }
}
```



1.5、浅复制与深复制

复制构造函数

```
student::student(student &s)
{
   cout<<"Copy Constructor";
   if(s.pName)
   {
     if(pName=new char[strlen(s.pName)+1])
          for(int i=0;i<=strlen(s.pName))
            pName[i]=s.pName[i];
       cout<<pName<<endl;
   }
   else  pName=nulptr;
}
```

if(pName = new (std::nothrow) char[strlen(s.pName)+1])



1.5、浅复制与深复制

```
student & student::operator=(student &s){
    cout<<"Copy Assign operator";
    if(this!=&s)  {
        if(pName) delete[ ] pName;
        if(s.pName)
        {
          if(pName=new char[strlen(s.pName)+1])
              for(int i=0;i<=strlen(s.pName))
                 pName[i]=s.pName[i];
          cout<<pName<<endl;
        }
        else pName=nulptr;
     }
    return *this;
}
```

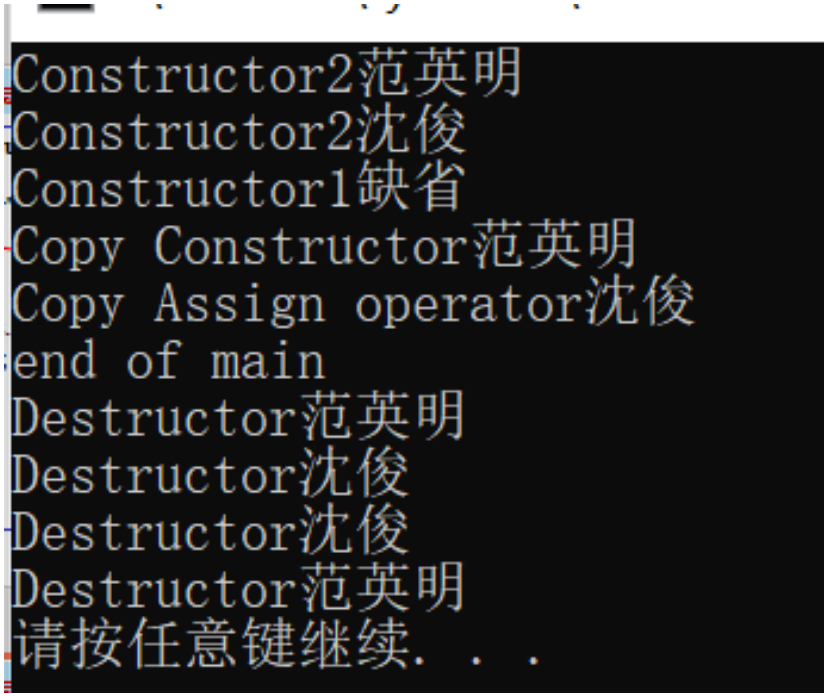
赋值运算符重载

如原来已分配，应先撤销，再重分配



1.5、浅复制与深复制

```
int main(void)
{
        student s1("范英明"),s2("沈俊"),s3;
        student s4=s1;
        s3=s2;
    return 0;
}
```



By 王文博

链 表

东南大学 信息科学与工程学院
王文博 2026

2、链表

线性表：

- 顺序表：元素的插入删除需要多次移动

- 链表：插入和删除时，不需要移动表中数据

By 王文博

线性表

顺序表

- 创建
- 输出
- 查找
- 插入
- 删除
- 求长度
- 判表空
- 判表满

元素的插入删除需要多次移动

常见操作

链表

- 创建
- 输出
- 查找
- 插入
- 删除
- 求长度
- 判表空

元素的插入删除不需要移动

东南大学 信息科学与工程学院
王文博 2026

2.1、单链表基本算法

- 单链表中每个元素占用一个结点(node):



结点包含两个域：

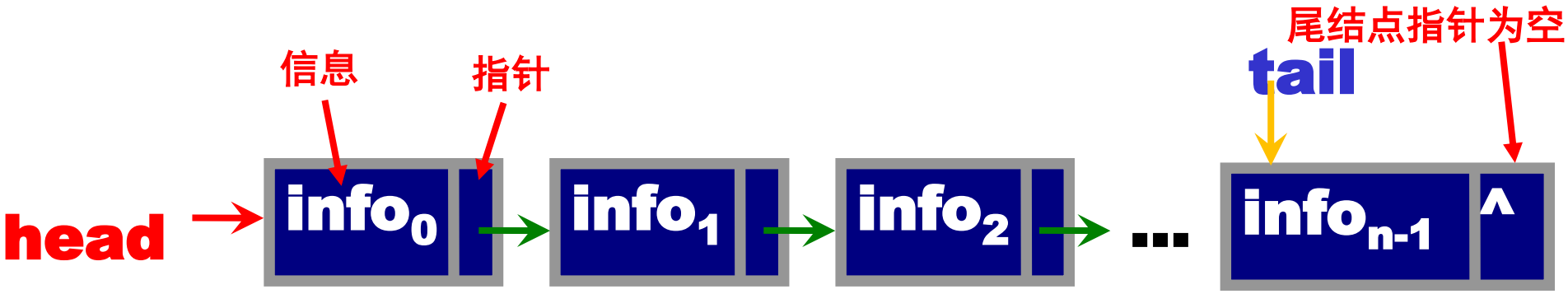
◆一个域存放数据元素，

◆一个域存放指向下一个结点的指针。

```
typedef int Datatype;  //数据为整型
struct node
{
   Datatype info;
   node *link;
};
```

允许成员是自身指针类型，但绝不允许是自身类型变量

2.1、单链表基本算法



单链表结构

- 首结点的地址存放于表头指针head，head

丢失则链表无法访问，且内存泄漏！

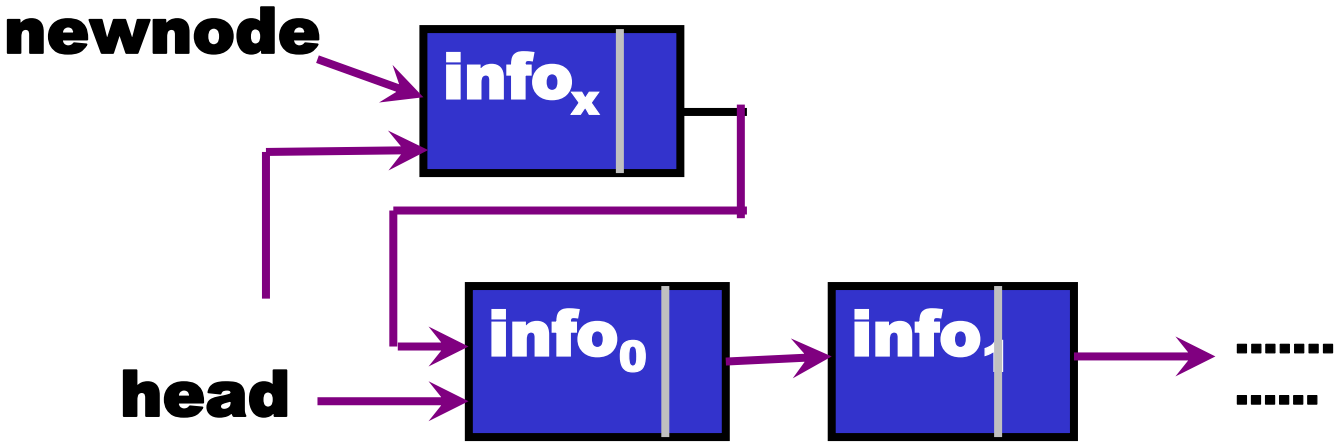
2 .1.1、单链表基本算法-插入

Case a)插在链首：

newnode→link=head；

head =newnode；

注意：链表操作次序非常重要!!



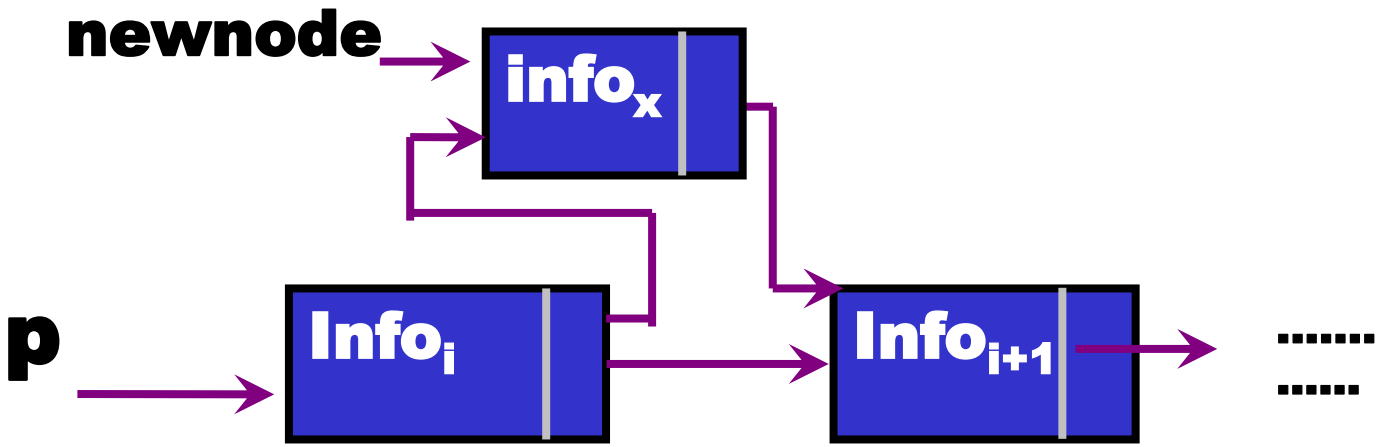
2.1.1、单链表基本算法-插入

Case b)插在中间：

newnode→link=p→link；

p→link=newnode；

注意：链表操作次序非常重要!!



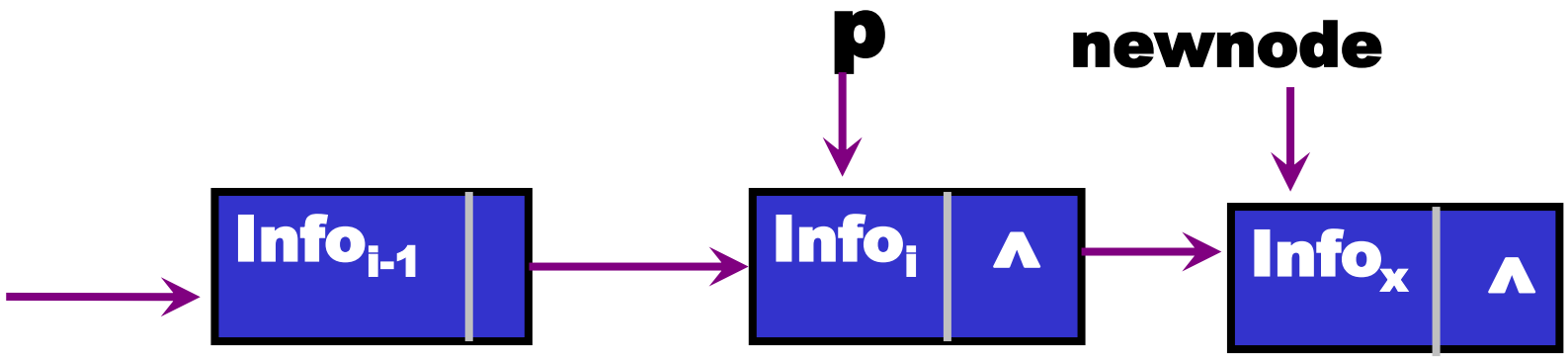
2.1、单链表基本算法-插入

注意：链表操作次序非常重要!!

Case c)插在队尾：

newnode→link=nulptr；//p → link

p→link=newnode ；



2.1、单链表基本算法-插入

王文博

Case a)插在链首：

newnode→link=head；

head =newnode；

Case b)插在中间：

newnode→link=p→link；

p→link=newnode；

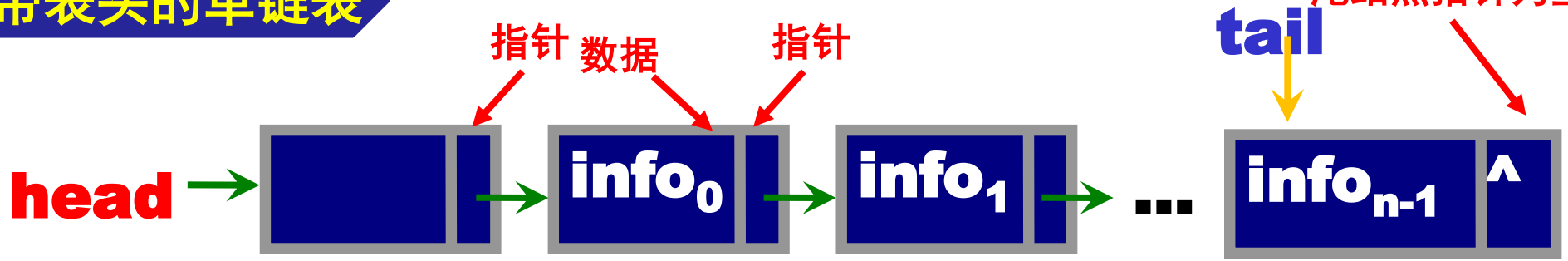
Case c)插在队尾：

newnode→link=p → link

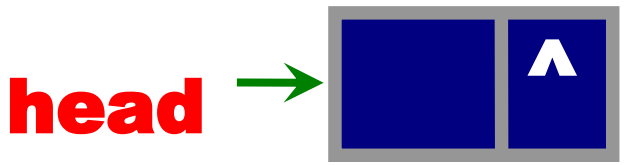
p→link=newnode ；

2.1、单链表基本算法

带表头的单链表



带表头结构的链表



带表头结构的空链表

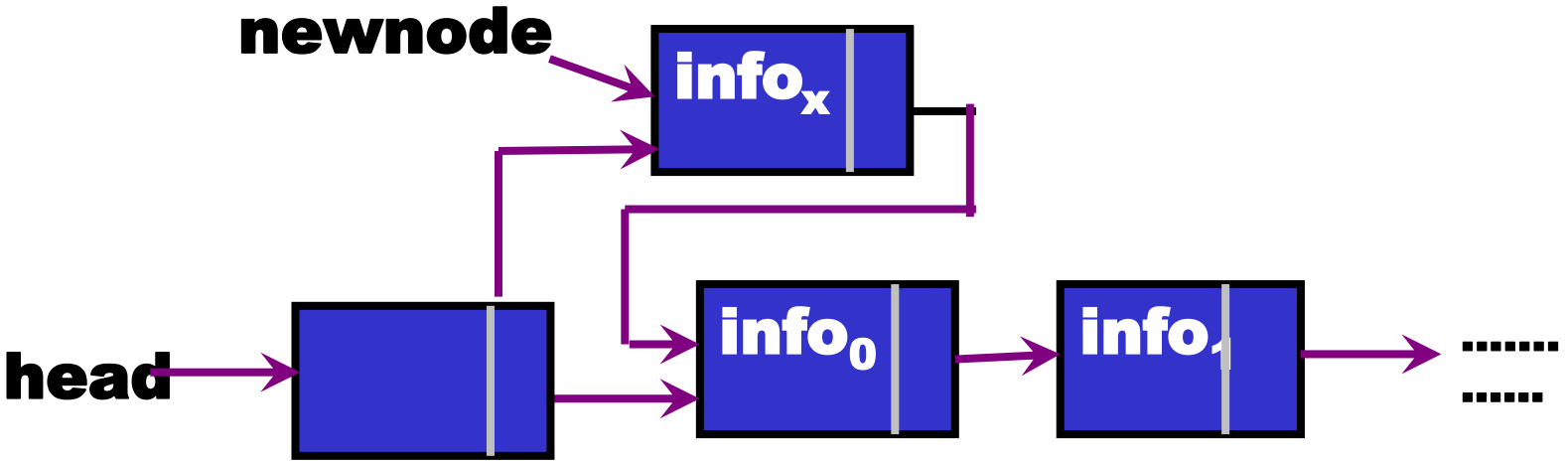
2.1.1、单链表基本算法-插入

带表头的单链表

Case d)插在链首（带表头链表）：

newnode→link=head→link;

head →link =newnode;



2.1.1、单链表基本算法-插入

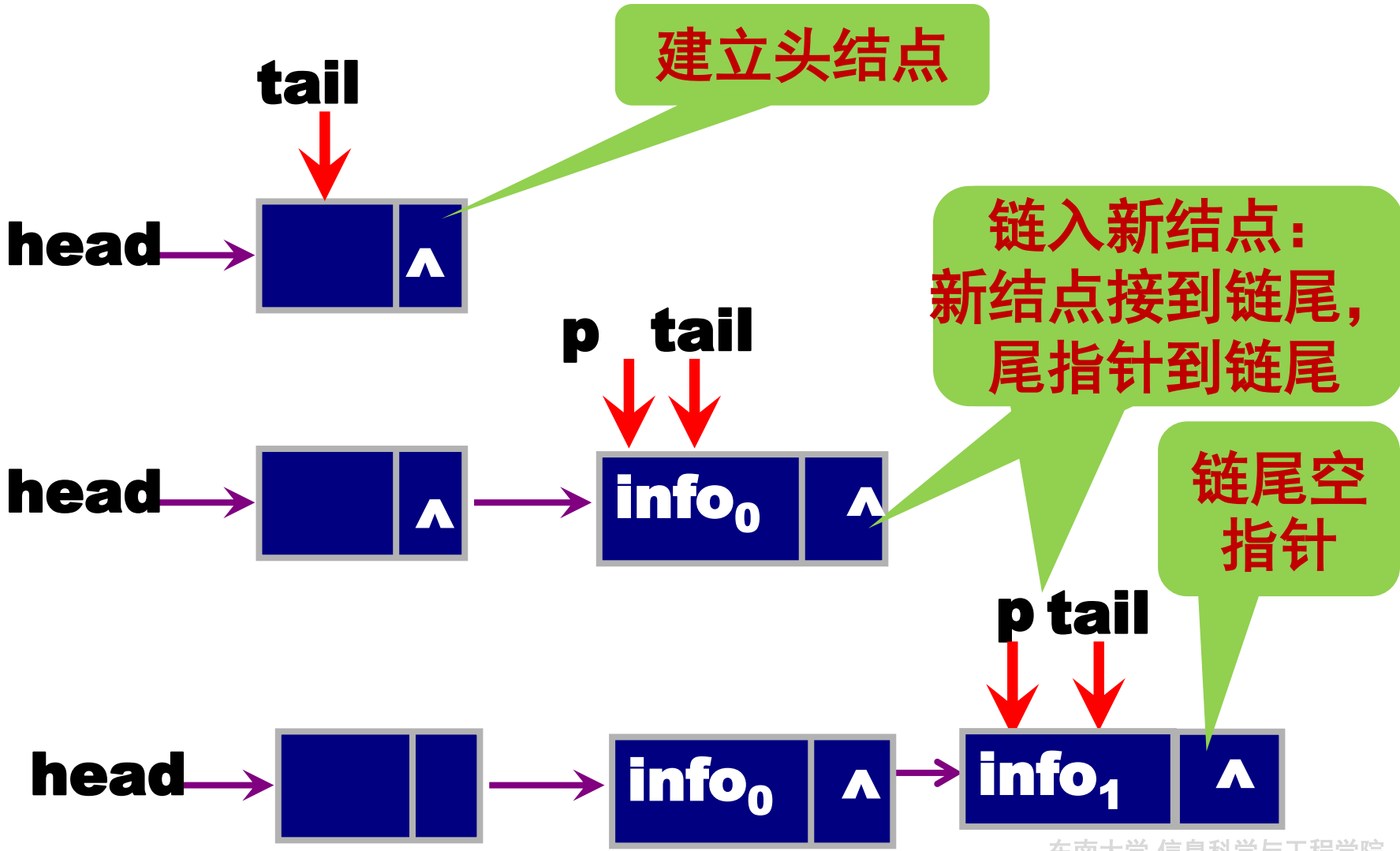
一般算法：

将数据x添加到中，在单链表(带表头)的p结点后插入：

```
void insert(node *p, Datatype x)
{
    node *newnode=new node;
    newnode->info=x;
    newnode->link=p->link;
    p->link=newnode;
}
```

2.1.2 、链表的生成算法-向后生产法

向后生成法



2.1.2、链表的生成算法-向后生产代码

```
node *createdown()
{  Datatype data;
     Node*head,*tail,*p;
     head=new node;
     head->link=NULL;
     tail=head;
     while(cin>>data)
     {    p=new node;
          p->info=data;
          p->link=tail->link;
          tail->link= p;
          tail=p;
     }
     return
     }
```

三个指针

建立链表头结点

链尾

每输入一个数申请一个结点

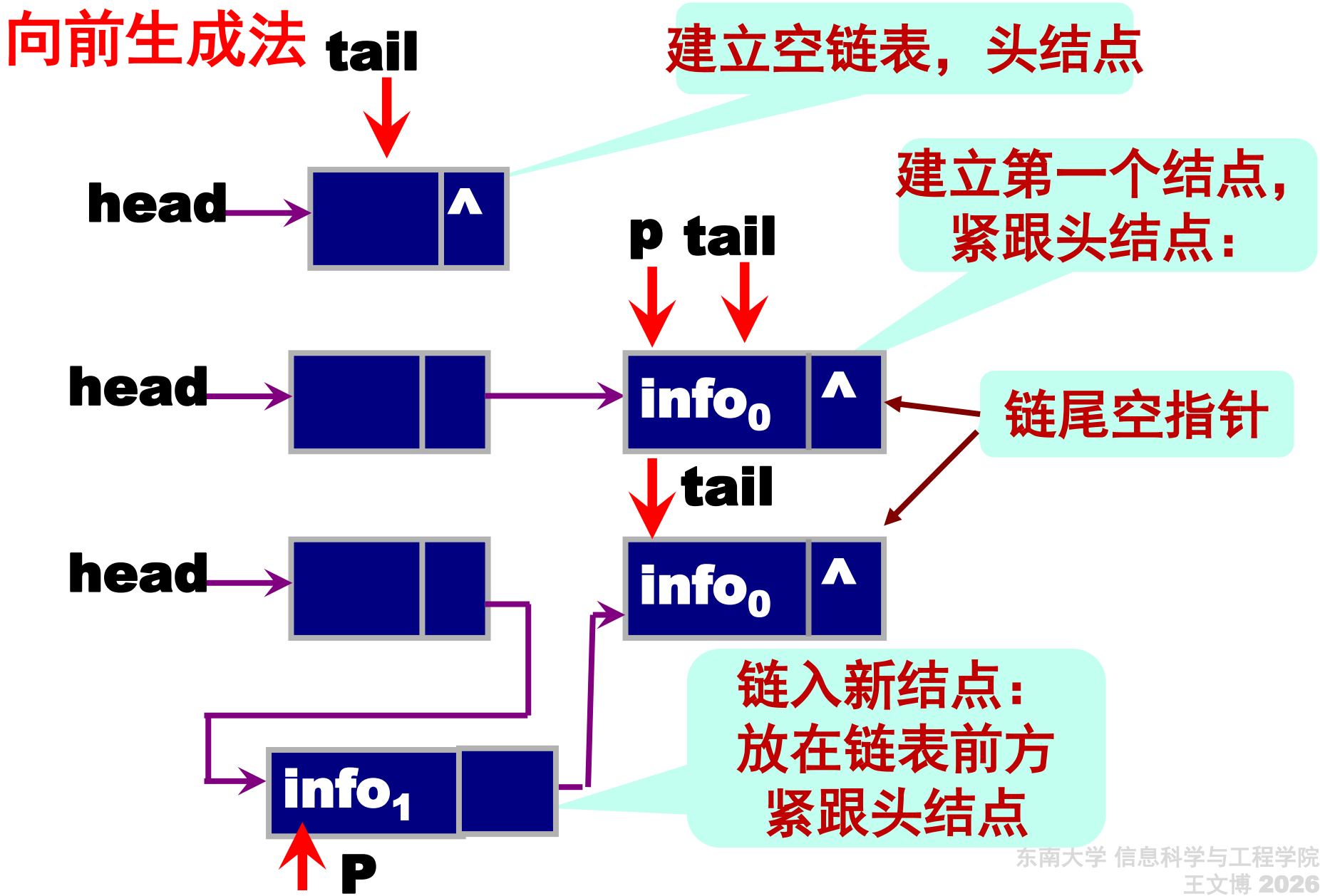
添入数据

新结点接到链尾

尾指针到链尾

返回头指针

2.1.3 、链表的生成算法-向前生产法



2.1.3、链表的生成算法-向前生产法代码

```
node *createup()
{
    node *head,*p,*tail;
    Datatype data;
    head=new node;
    head->link=NULL;
    tail=head;
    while(cin>>data)
     {
        p=new node;
        p->info=data;
        p->link= head->link ;
        head->link=p;
        if (tail==head) tail=p;
      }
    return head;
}
```

建立链表头结点

链尾空指针

新结点放在链表前方

头结点放新结点之前

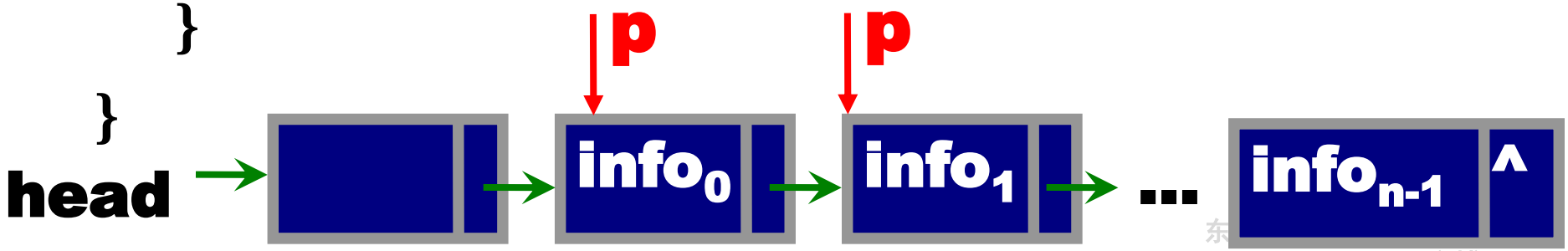
第一个结点插入需要调整tail

返回头指针

2.1.4、链表的基本算法-遍历链表

链表中数据访问通过工作指针实现，从表头开始逐一指向后续结点，直至NULL.

```
void print(node *head)
{    node *p=head->link;
      while(p!=NULL)
     {
      cout<<p->info<<'\t';
      p=p->link;
     }
}
```



王文博 2026

2.1.5、链表的基本算法-查找

```
node *find(node *head, Datatype data)
{
    node *p=head->link;
    while(p!=NULL&&p->info!=data)
      p=p->link;
    return p;
    }
```

返回值为指针p，指向链表中找到的结点。

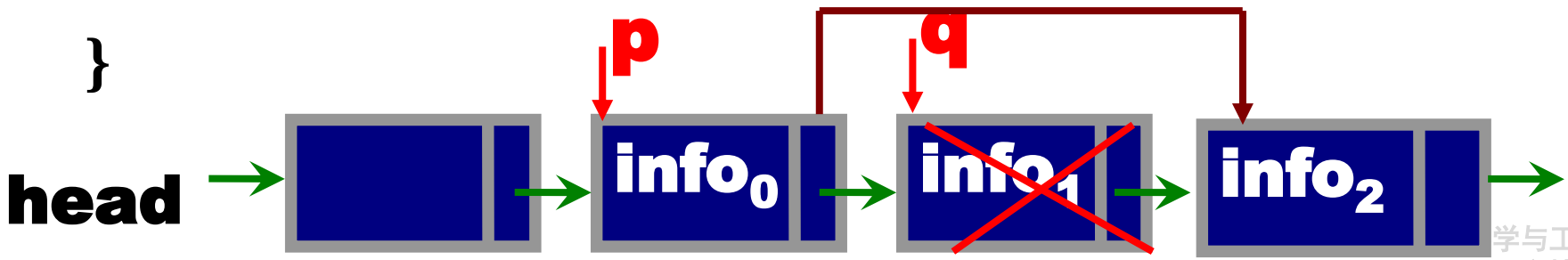
p为空则表示未找到

2.1.6、链表的基本算法-删除结点

```
void del (node *p)  //删除结点p后面的一个结点
{
    node *q;
    q=p->link;
    if( q!=NULL){
    p->link=q->link;
     delete q;
   }
}
```

q指向*p后面的结点

如果要把该结点移入另一个链中，可将q返回



学与工程学院
王文博 2026

2.1.7、链表的基本算法-单链表模板

```
template<typename T>class List; //提前引用声明
```

定义结点类模板

```
template<typename T>class Node
{
    T info;                    //数据域
    Node<T> *link;             //指针域,<>中是参数名表,类模板实例化为类
public:
    Node();                    //生成头结点的构造函数
    Node(const T & data); //生成一般结点的构造函数
    void InsertAfter(Node<T>* p); //在当前结点后插入p节点
    Node<T>* RemoveAfter();  //删除当前结点的后继结点
    friend class List<T>;  //以List为友元类
};
```

2.1.7 、链表的基本算法-单链表模板

```
template<typename T>Node<T>::Node()
{ link = NULL; }
```

生成头结点的构造函数

杜绝野指针！！

```
template<typename T>
Node<T>::Node(const T & data)
{
    info = data;
    link = NULL;
}
```

生成一般结点的构造函数

复制构造函数？



2.1.7 、链表的基本算法-单链表模板

```
template<typename T>
void Node<T>::InsertAfer(Node<T>* P)
 {
    P->link = link;
    link = P;
}
```

在当前结点后插入一个结点

```
template<typename T>
 Node<T>* Node<T>::RemovAfter()
{
    Node<T>* tempP = link;
    if (link == NULL) tempP = NULL;
    else
        link = tempP->link;
    return tempP;
}
```

删除当前结点的后续结点，返回该结点备用



2.1.7、链表的基本算法-单链表模板

定义链表类

```
template<typename T>class List
{
    Node<T> *head,*tail; //链表头指针和尾指针
public:
    List();  //构造函数，生成头结点(空链表)
    ~List(); //析构函数
    void Empty(); //清空链表，只余表头结点
    Node<T>* Find(const T & data) const; //搜索结点，返回地址
    int Length() const;    //计算单链表长度
    void PrintList() const;   //打印链表的数据域
    void InsertFront(Node<T>* p);  //可用来向前生成链表
    void InsertRear(Node<T>* p);   //可用来向后生成链表
    void InsertOrder(Node<T> *p);  //按升序生成链表
    Node<T>*CreatNode(const T & data);   //创建结点(孤立结点)
    Node<T>*DeleteNode(Node<T>* p); //删除指定结点
};
```



2.1.7 、链表的基本算法-单链表模板

构造函数与析构函数

```
template<typename T> List<T>::List()
{
   head = tail = new Node<T>();
}
template<typename T> List<T>::~List()
 {
   Empty(); //从表中删除数据节点
   delete head;//释放头结点
}
```

```
template<typename T>
void List<T>::Empty()
 {
   Node<T>* tempP;
   while (head->link != NULL) {
      tempP = head->link;
      head->link = tempP->link;
      delete tempP;
   }
   tail = head;
}
```

2.1.7 、链表的基本算法-单链表模板

搜索结点，返回地址

```
template<typename T>Node<T>* List<T>::Find(T data)
 {
    Node<T>* tempP = head->link;
    while (tempP != NULL && tempP->info != data)
            tempP = tempP->link;
    return tempP;
}
```

第一个数据结点

如果未找到，返回的是NULL



2.1.7 、链表的基本算法-单链表模板

计算单链表长度

```
template<typename T> int List<T>::Length()
 {
     int count = 0;
     Node<T>* tempP = head->link;
      while (tempP != NULL)
     {
          tempP = tempP->link;
          count++;
      }
    return count;
}
```



2.1.7、链表的基本算法-单链表模板

```
template<typename T>
void List<T>::InsertFront(Node<T> *p)
{
    p->link = head->link;
    head->link = p;
    if (tail == head) tail = p;
}
```

向前生成链表

```
template<typename T>
void List<T>::InsertRear(Node<T>* p)
{
    p->link =tail->link;
    tail->link = p;
    tail = p;
}
```

向后生成链表



2.1.7 、链表的基本算法-单链表模板

```
template<typename T> void List<T>::InsertOrder(Node<T>* p)
{
    Node<T>* tempP = head->link ;                    |;
    while (tempP != NULL) {
        if (p->info < tempP->info) break;
        tempQ = tempP;
        tempP = tempP->link;
    }
    tempQ->InsertAfer(p);
    if (tail == tempQ) tail = tempQ->link;
}
```

查找正确的插入位置

！！！次序不能反

按升序生成链表



2.1.7 、链表的基本算法-单链表模板

创建结点（孤立结点）

```
template<typename T>
 Node<T>* List<T>::CreatNode(T data)
{
    Node<T>* tempP = new Node<T>(data);
    return tempP;
}
```



2.1.7 、链表的基本算法-单链表模板

删除指定结点

```
template<typename T>
Node<T>* List<T>::DeleteNode(Node<T>*p)
 {
    Node<T>* tempP = head;
    while (tempP->link != NULL && tempP->link != p)
          tempP = tempP->link;
    if (tempP->link == tail)  tail = tempP;
    return tempP->RemovAfter();
}
```

从链表中删除结点，并返回结点的地址



2.1.7 、链表的基本算法-单链表模板

单链表类型模板应用示例1——int型数据链表

随机产生N个不同的整数，以这些整数为结点数据：

1)生成两个链表：一个向前生成list1，一个向后生成list2

2)输出两个表。

3) 在链表list1和list2中查找一个整数并删除它，输出链表。

4) 将上面list1中删除的结点，插入list2链表尾部，输出该表

5) 清空list1链表，再把随机产生的N个整数，按升序生成链表并输出

2.1.7、链表的基本算法-单链表模板

```
#include "singlelinklist.h"
int main() {
        Node<int>* P1,*P2;
        List<int> list1, list2;
        int *a, i, j;
        out << "请输入链表结点个数" << endl;
        int N=0; cin >> N;
        a=new int[N];
        srand(0);
        for (i = 0; i < N; i++) {
                a[i]=rand()%100+1;
                for(j=0;j<i;j++)        {
                    if ( a[i]==a[j] ) {  i--; break; }
                }
        }
```

结点指针，数据类型int

链表list1, list 2;

随机产生N个整数

2.1.7 、链表的基本算法-单链表模板

```
 for (i = 0; i < N; i++)
{
        P1 = list1.CreatNode(a[i]);
        list1.InsertFront(P1);
        P1 = list2.CreatNode(a[i]);
        list2.InsertRear(P1);
 }
 list1.PrintList();
 cout << "list1长度： " << list1.Length() << endl;
list2.PrintList();
cout << "list2长度： " << list2.Length() << endl;
```

向前生成list1

向后生成list2

南京大学 信息科学与工程学院
王文博 2026

2.1.7、链表的基本算法-单链表模板

```
cout << "请输入一个要求删除的整数" << endl;
cin >> j;
P1 = list1.Find(j);
if (P1 != NULL) {
        list1.DeleteNode(P1);
        list1.PrintList();
        cout << "list1长度： " << list1.Length() << endl;
}
else cout << "未找到" << endl;
if (P2 = list2.Find(j)
{
        list2.DeleteNode(P2);
        delete P2;
        list2.PrintList();
        cout << "list2长度： " << list2.Length() << endl;
 }
else cout << "未找到" << endl;
```

释放被删除结点的空间

2.1.7 、链表的基本算法-单链表模板

```
cout<<"插入表1中删除的结点后，表2为： "<<endl;
list2.InsertRear(P1);
list2.PrintList();

list1.MakeEmpty();
for (i = 0; i < 15; i++)
{
        P1 = list1.CreatNode(a[i]);
        list1.InsertOrder(P1);
}
list1.PrintList();
delete[] a;
return 0;
}
```

将P1指向的结点插入list2中

清空list1

升序创建list1

代码示例